UNIVERSIDAD D SEVILLA

Tema 2. Integridad de la Información

Dr. Rafael Ceballos Guerrero
Grupo de Investigación **Quivir**,
Tecnologías Inteligentes y de Seguridad de los Sistemas de Información
Departamento de Lenguajes y Sistemas Informáticos
Universidad de Sevilla





Implementación de la Confidencialidad.
Password Integrity and Confidentiality

- En la autenticación de usuarios: el password presentado por un usuario se le aplica una función hash y se le compara con el hash almacenado.
- Para defenderse de ***dictionary attacks*** y ataques pre-computados ***rainbow table,*** los password se suelen concatenar con un valor aleatorio, no-secreto ***salt*** antes de aplicar la función hash
- El salt ayuda contra los ataques con diccionarios al añadir aleatoriedad, y contra las tablas rainbow al extender la longitud y con ello la complejidad de la contraseña
- Si dos usuarios generan una misma contraseña, su hash no será idéntico usando salt

Implementación de la Confidencialidad.
Password Integrity and Confidentiality

- El ***salt*** es una secuencia de bytes que se añade al password antes de generar el resumen (salt recomendado >=8 bytes)
- Usar la **estrategia de Salting:**
  - **Salt fijo**, secuencia de bytes que se digiere junto a cada password. Se guarda oculto y es valor añadido de seguridad
  - **Salt variable**, generalmente la opción más segura (mejor si es aleatorio). Generado y procesado separadamente de para cada password que se digiere y permite que cada **password almacenado se encuentre desacoplado de los otros**
- **Problemas de un salt fijo**: si un atacante obtiene este salt, la seguridad de toda la base de datos decrecerá de forma sustancial
- Una solución sería mezclar salt fijo y variable, parte fija y parte aleatoria, y se almacena la parte aleatoria y el resumen

Implementación de la Confidencialidad.
Password Integrity and Confidentiality

- ***Key stretching functions***, tales como PBKDF2, Bcrypt o Scrypt, tipicamente son invocaciones repetidas del hashing seguro para incrementar el tiempo requerido en los ***brute force attacks*** de la disgestión de los password almacenados.
  - **El iteration count** representa el número de veces que la hash function con que la digestión se aplica a los resultados previos.

```
key = hash(password)
 for 1 to 65536 do
     key = hash(key)

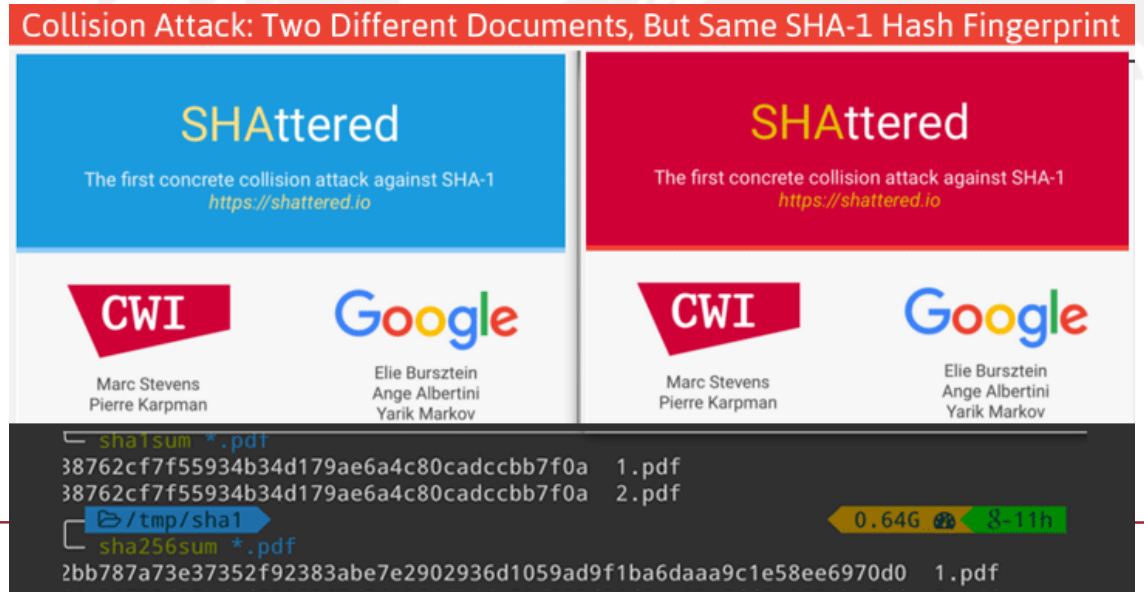
key = hash(password+salt)
for 1 to 65536 do
     key = hash(key+salt)
```

Implementación de la Integridad. Colisiones

- Es matemáticamente imposible que no existan colisiones
  - El número potencial de posibles entradas es mayor que el número de salidas que puede producir un resumen
- Una ataque por colisiones es encontrar dos entradas arbitrarias cuyos resúmenes coincidan
  - Sería más complejo sería poder encontrar una entrada (inverse image) cuyo resumen coincida con el de una entrada concreta
- Una ataque por colisiones sería factible si existe un método capaz de encontrarla en menos tiempo que un ataque de cumpleaños de fuerza bruta
  - Paradoja del cumpleaños: dado un conjunto de 23 personas, hay una probabilidad del 50,7% de que al menos dos personas de ellas tengan el mismo cumpleaños; y para 57 o más personas la probabilidad > 99%

Implementación de la Integridad. Colisiones

- El cifrado SHA-1 ya no es seguro: Google lo ha roto después de 22 años
  - Grupo de investigadores de Google y del CWI de Amsterdam han logrado con éxito un **ataque de colisión** en SHA-1
  - Problema: **han conseguido modificar un mensaje, y el resumen obtenido para el original y el modificado son iguales.**
  - 100.000 veces más rápido que un ataque por fuerza bruta (aún no viable)



Implementación de la integridad en Storage.
File Integrity Monitoring Systems

- Objetivo:
  - **Ser capaces de detectar cualquier modificación de ficheros críticos del sistema.** Usan funciones resumen (SHA-256, SHA-512)
- ¿Qué tipo de ataques se pueden detectar?
  - **Ataques de puertas traseras**
  - **Entrada adicional en el fichero de contraseñas o**
  - **Un /bin/login que permite el acceso ante cierto nombre de usuario no registrado**
- ¿Problemas de este tipo de sistemas?
  - **Resumen inicial. ¿En quién confío?**
  - **Nivel de privilegio.**
  - **Mantener los digest fuera del alcance del atacante en una base de datos y en dispositivos de solo lectura.**
  - **Evitar falsos positivos: Tener en cuenta cuando se actualizan estos ficheros y se parchean los sistemas**
- Herramientas: **Tripwire** (el más usado), **VeriSys, Cimtrak, Nitko…**

- **Tripwire File Integrity**
  - Para sistemas de ficheros Linux, UNIX y Microsoft Windows
  - Tripwire Academic Source Release for Linux is available free of charge
- **VeriSys**
  - Para Windows Server, Windows SQL, IIS, Apache
  - Monitoriza propiedades de ficheros, propiedades de los registros de claves,
  - Rapidez de análisis de la integridad y elaboración de informes.
  - Puede enviar correo cuando se acaba la verificación.
  - Administración centralizada, controla a todos los agentes desplegados

- **Nikto escaner Open Source (GPL) para servidores web**:
  - Incluye comprobación de ficheros/CGI potencialmente peligrosos,
  - Comprueba versiones de servidores Web, y problemas específicos de versiones de más de 250 servidores.
  - Comprueba las configuraciones del servidor tales como la presencia de ficheros de indexación múltiples, opciones del servidor HTTP, etc...
  - Escanea elementos y plugins que se actualizan frecuentemente.

- WFP o “Windows Protection File”
  - Se guarda una copia de los archivos de sistema (DLL, EXE, SYS, OCX etc.) en **/windows/system32/dllcache**, y si observa que variación alguna del checksum o borrado
  - Sólo algunos components del Sistema como Package Installer (Update.exe) o Windows Installer (Msiexec.exe) pueden reemplazar las firmas de los ficheros (carpeta protegida)
  - Todos los cambios usando otros métodos serán revertidos usando las copias originales de la cache (o la red si hace falta, o se le pide al usuario que introduzca el disco de instalación)

- **Lenguajes de programación**:
  - Paso por valor y por parámetros cuando se llaman a las funciones. ¿Y en Java?
  - ***assert*** entre las instrucciones de procesamiento de los datos.
    - assert accontBalance >= 0;

- **Integridad en Bases de datos** asegura que los datos introducidos en ella son exactos, válidos y consistentes.
  - Uso de ***restricciones de integridad y reglas de*** data validation se deben de satisfacer antes de permitir el cambio de la base de datos.
  - Tres tipos de restricciones de integridadd:
    - Entity Integrity, no permitir múltiples filas con la misma identidad.
    - **Domain integrity**, restringir los datos a los definidos en el tipo
    - **Referential integrity** dos tablas; required that the foreign key is equal to the candidate key in some row of the primary table, or else have no value (valor NULL).
  - **¿Cómo se implementan?**

DB Triggers

Importancia de la Integridad

- Áreas importantes de interés de la integridad de la información están en:
  - Antivirus, Kaspersky por ejemplo lo utiliza para detectar malware
  - Descarga de software, distribuciones Linux por ejemplo
  - En el entorno Blockchain permite el cerrado de bloques y el estampado de documentos

Implementación de Integridad. Otras utilidades de las funciones hash seguras

- **Data Authentication**
  - Para probar que dos elementos son idénticos. Una cuestión importante para el Análisis Forense salvaguardar la integridad de sus evidencias o pruebas digitales. Esto se hace via Cadena de Custodia, Documentación, y **Hash values**.
- **Data Reduction, File Identification**
  - Es más fácil comparar hash values que ficheros completos
  - Encontrar una aguja en un pajar

Implementación de Integridad. Otras utilidades de las funciones hash seguras

- **Ejemplo: Dropbox**
  - Dropbox bloqueó a uno de sus usuarios porque compartía contenido protegido por la ley
  - Mucha gente pensaba que Dropbox accedía a las cuentas de los usuarios, a pesar de que la propia compañía declara lo contrario
  - Los poseedores de los derechos de autor sobre el archivo compartido crearon un resumen (función hash)
  - Dropbox añadió este resumen a la lista negra de contenidos protegidos
  - Cualquier intento de compartir un archivo de la lista negra es bloqueado por defecto
- **Ejemplo: Github**
  - De cada objeto Git se guarda su hash SHA-1

- **Recordar muchos passwords es una difícil tarea,**
  - Passwords son mas fáciles de recordar cuando uso un algoritmo de hashing (Yo cambio el que uso periódicamente), y entonces usamos los primeros (o últimos o mitad) 10 caracteres como password.
    - Puedes tener el fichero de texto en tu smart phone solamente tienes que recordar el algoritmo de hashing y los caracteres usados.

Temas Relacionados. Limpieza segura y recuperación

- **Limpieza segura de soportes**
  - **Eraser**
  - **Bcwipe**
  - **Data Shredder** (conocido como CBL Data Shredder)
- **Recuperación en soportes**
  - **File Scavenger®**, un programa para recuperar archivos y datos eliminados en sistemas Windows
  - **Photorec:** Software libre